

Clases, métodos y constructores

No.	Respuesta	Explicación:
1	b. ...	Los tres puntos ... se utilizan para declarar un parámetro varargs. Esto permite enviar varios valores al método y dentro del método se puede tratar como un arreglo. Además, el parámetro varargs debe ser el último de la lista. [] solamente declara un arreglo normal y varargs no es una palabra reservada de Java.
2	d. this()	La respuesta es correcta porque this() sirve para llamar a otro constructor de la misma clase.
3	g. X only.	La clase X no declara ningún constructor, así que Java crea automáticamente uno sin argumentos. Y sí tiene un constructor sin argumentos, pero fue escrito manualmente por lo que técnicamente no se considera el constructor por defecto. Z solamente tiene Z(int), por lo que no tiene un constructor por defecto.
4	c. Compilation fails	Java necesita que la llamada coincida con la cantidad y los tipos de parámetros que tiene el método. Entonces, si el método espera tres argumentos y nosotros solo mandamos dos, el compilador marca un error antes de ejecutar el programa.
5	d. Un objeto inmutable no permite ningún cambio en su estado después de ser creado.	Un objeto inmutable es uno que, después de crearse, no puede cambiar su estado.
6	a. hello	Primero str se inicializa con "default", pero después se ejecuta el constructor X(String s), que cambia str por "hello". Por eso cuando se llama a print(), el valor que tiene la variable en ese momento es "hello".
7	b. 12	Java pasa los argumentos por valor. Eso significa que el método m() recibe una copia de i. Entonces i += 7 cambia solamente la copia dentro del método y no la variable original que está en main. Por eso al final sigue valiendo 12.
8	c. 20	a es un int, que es un tipo primitivo, y Java pasa una copia de su valor al método. Aunque llamemos al método dos veces, las dos veces se modifica una copia y no la variable a de main. Por eso sigue valiendo 20.
9	d. Display with no arguments Display with integer: 5 Display with string: Hello	Al llamar display() se usa el método sin parámetros, al llamar display(5) se usa el que recibe int y al llamar display("Hello") se usa el que recibe String. Por eso se imprimen las tres líneas correspondientes.
10	e. price.price = 4;	La variable local price es de tipo Simple, es decir, contiene una referencia a un objeto. Por eso no podemos hacer price =

		4, porque estaríamos intentando meter un int dentro de una variable que espera un objeto Simple. Con price.price = 4 estamos accediendo al atributo price del objeto y asignándole el valor 4.
11	b. Bob's Name: Jian	Cuando hacemos jian = bob, no estamos copiando el objeto. Estamos haciendo que jian y bob apunten al mismo objeto. Entonces cuando hacemos jian.name = "Jian", estamos modificando ese único objeto y bob.name también pasa a ser "Jian".
12	b. Compilation fails	Un parámetro varargs debe ser siempre el último parámetro del método.
13	b. Hello	String es inmutable y Java pasa los argumentos por valor. Dentro de op1, la operación s += "Java" crea un nuevo String y cambia solamente la referencia local s. La variable msg que está en stringTest() sigue apuntando a "Hello", por eso se imprime Hello.
14	a.El constructor por defecto siempre invoca el constructor sin argumentos de la superclase. d.El constructor por defecto se crea solo cuando la clase no tiene ningún constructor definido.	La a es correcta porque el constructor por defecto generado por Java llama automáticamente a super() sin argumentos. La d es correcta porque el compilador solamente crea ese constructor automáticamente cuando la clase no tiene ningún constructor. Las otras opciones no son porque un constructor no siempre es public, y tampoco hereda los argumentos de los constructores de la superclase.
15	c.Class2 has-a v2. d.Class3 has-a v1. f.Class2 has-a Class1.	Class2 tiene una variable v2, por lo que podemos decir que Class2 has-a v2. También Class2 tiene un atributo de tipo Class1, así que Class2 has-a Class1. Finalmente, Class3 tiene un Class2, que a su vez tiene un Class1, y este contiene v1, por lo que Class3 tiene acceso a esa relación de forma indirecta.
16	e. 6564	Primero el z de main vale 6, por eso se imprime 6. Dentro de doStuff() existe otro z local que vale 5, y doStuff2() cambia el atributo z del objeto a 4, no el z local de doStuff(). Después se imprime otra vez el 6 de main y finalmente el atributo myScope.z, que vale 4. Por eso el resultado es 6564.
17	a. Compilation fail	Primero, la declaración ScopeTest myScope new ScopeTest(); está incompleta porque falta el =. Además, myScope es una variable local de main, por lo que no puede utilizarse directamente dentro de doStuff(). Por esas razones el programa no puede compilar.

18	d.Constructor with argument called: Default No-argument constructor called Name: Default Hello:Default Constructor with argument called: John Name: John Hi:John	El constructor sin argumentos hace primero this("Default"). Eso significa que antes de ejecutar el resto del constructor, Java entra al constructor que recibe String y ahí imprime Constructor with argument called: Default. Después regresa al constructor sin argumentos y muestra No-argument constructor called. Luego se ejecutan display() y display("Hello") en ese orden. Con obj2 ocurre lo mismo siguiendo el orden de las llamadas solo omitiendo el constructor vacío, este entra directamente al que recibe un String.
19	c. Constructor with int argument: 10 No-argument constructor No-argument calculate method Calculate method with int argument: 5 Constructor with int argument: 20 No-argument calculate method Calculate method with int argument: 15	this(10) es la primera instrucción del constructor sin argumentos, así que primero se imprime Constructor with int argument: 10 y después No-argument constructor. Luego las llamadas a calculate() se eligen dependiendo de los argumentos, así que calculate() usa el método sin parámetros y calculate(5) usa el que recibe int. Obj2 solo entra al constructor que recibe un int lo demás se mantiene en el orden de obj1.
20	c.Constructor with name and age called Default constructor called Constructor with name and age called Constructor with name called Constructor with name and age called	Cuando se crea obj1, se llama al constructor sin argumentos, pero ese constructor primero ejecuta this("Unknown", 0). Por eso primero aparece Constructor with name and age called y después Default constructor called. Luego obj2 llama al constructor con String, que a su vez llama al constructor con String e int, y finalmente imprime Constructor with name called. obj3 va directamente al constructor con ambos parámetros.
21	a. Error de compilación	El constructor sin argumentos llama a this(10) y el constructor con int llama nuevamente a this(). Eso crea un ciclo: uno llama al otro y el otro vuelve a llamar al primero. Java detecta esta llamada recursiva entre constructores durante la compilación y no permite que el programa llegue a ejecutarse.
22	a. Compilation error due to recursive constructor calls	El constructor sin argumentos llama al constructor con int, y el constructor con int vuelve a llamar al constructor sin argumentos. Java detecta el problema al compilar y genera un error de recursive constructor invocation.
23	f. double	Java elige primero las sobrecargas que pueden utilizarse sin hacer boxing ni varargs. Un float puede convertirse automáticamente a double, así que doIt(double) es válido en esa primera fase. Float necesita boxing y float... necesita

		varargs, y esas opciones se consideran después. Por eso gana <code>doIt(double)</code> .
24	b. 3	Compilan tres formas: <code>main(String... a)</code> , <code>main(String...a)</code> y <code>main(String[]... a)</code> . Las dos primeras son varargs normales y la tercera representa un varargs de arreglos <code>String[]</code> . En cambio, <code>String. a</code> tiene una sintaxis incorrecta y <code>String...[]</code> a tampoco es válido.